Terraform Hands-on Exam and Q&A

Section 1: Q&A (20 Questions)

- **Terraform Fundamentals (5 questions)**
    - What is Terraform and how does it differ from other IaC tools?

      Terraform is an IaC tool by HashiCorp, allowing infrastructure definition and management through a declarative process simply stipulate the outcome, and Terraform determines the steps needed to achieve it.
      Unlike Ansible, Chef, and Puppet, which focus on configuring existing servers Terraform provisions infrastructure and manages it. It also created a more predictable environment by managing changes with a state file.
      Terraform special because it is modular, is cloud agnostic (supports AWS, Azure, GCP, etc.), and automatically generates a plan for infrastructure changes (with terraform plan) before executing them, which is rare for other tools

    - Explain Terraform's declarative nature and state management.

      Terraform operates declaratively, so you don't have to write detailed instructions. rather, Terraform determines the necessary steps to get your desired infrastructure state, such as "I need two servers and a database."
      Terraform uses a state file to record what has already been built in order to keep track of everything. This keeps Terraform from unintentionally recreating items you already own by letting it know what is there, what has changed, and what needs updating.

    - What is the purpose of the Terraform provider?

      A Terraform provider is like an adapter that helps Terraform communicate with different platforms—AWS, Azure, Google Cloud, and more. It knows how to create, update, and delete resources for each service. Without providers, Terraform wouldn’t know how to manage infrastructure. That’s why every project starts by specifying the providers it needs

- How does Terraform handle dependency resolution?

  Terraform automatically figures out the order in which resources should be created, updated, or deleted based on their dependencies. It does this by analyzing resource references in the configuration.
  For example, if an EC2 instance depends on a VPC, Terraform ensures the VPC is created first. It builds a dependency graph and executes tasks in the right order. You can also use explicit dependencies with “depends on” when needed. This smart handling prevents errors and ensures smooth infrastructure changes.

- What are the key components of a Terraform configuration file?

  A Terraform configuration file is made up of several key components. Providers define which cloud or service provider you’ll use, like AWS or Azure. Resources specify the infrastructure elements you want to create, such as EC2 instances or databases. Variables allow you to set flexible input values, making the configuration reusable. Outputs display important values, like instance IPs, after Terraform applies the changes. Finally, Modules group related resources together for reuse in different projects. These components work together to help you define and manage your infrastructure efficiently.

- **State Management & Backend Configuration (3 questions)**

  - Explain the difference between `terraform refresh`, `terraform plan`, and `terraform apply`.

    The terraform refresh command updates your local state file to reflect any changes made directly in your infrastructure, but it doesn’t change anything. When you run terraform plan, it shows you a preview of what Terraform will do if you apply changes basically, it helps you see what will happen before anything is modified. And terraform apply takes the plan and carries out the changes to your infrastructure, making them real.

  - What is the difference between local and remote backends?

    The main difference between local and remote backends in Terraform is where the state file is stored. A local backend saves the Terraform state file on your local machine, which makes it suitable for small, personal projects. However, this can be risky for collaboration since others can’t easily access the state file. A remote backend, on the other hand, stores the state file in a remote location, like an S3 bucket or Terraform Cloud. This is better for teams because it allows for

centralized state management, versioning, and locking, preventing conflicts and improving collaboration.

- How can you prevent state corruption when multiple engineers work on the same infrastructure?

  you should use remote backends that support state locking. With services like Terraform Cloud, S3 with DynamoDB for state locking, you ensure that only one engineer can modify the state at a time. This prevents simultaneous changes that could corrupt the state file. Additionally, enforcing the use of workspaces and version control for configurations can help manage different environments and avoid conflicts.

- **Terraform Modules & Reusability (4 questions)**

  - What are the benefits of using Terraform modules?

    Terraform modules are great for keeping things simple and organized. Instead of repeating code for similar resources, you define it once in a module and use it whenever you need it. This makes your setup cleaner and easier to manage. modules also let you group resources in a way that makes sense, so things are easier to maintain. plus, they help you control changes better, since you can version them and easily roll back if something goes wrong.

  - Explain how to pass variables to a Terraform module.

    To pass variables to a Terraform module, you define the variables inside the module using the variable block. When calling the module, you pass the values for those variables in the module block. This allows you to customize the behaviour of the module based on your needs. You can also use variable files or environment variables for more flexibility in managing the values passed to the module.

  - What is the difference between `count` and `for_each`?

    The main difference between count and “for_each” in Terraform is how they manage resource creation. “count” is used when you want to create a specific number of identical resources. It works well when you know the exact number of resources you need, and each resource is similar.
    “for_each”, on the other hand, allows you to create resources from a collection, like a list or map. It’s more flexible because each resource can have different

values based on the collection, making it useful when the resources are similar but not identical.

- How do you source a module from a Git repository?

  to source a Terraform module from a Git repository, you specify the repository URL in the source argument of the module block. Terraform will fetch the module directly from the Git repository, and you can target a specific branch, tag, or commit by adding a reference in the URL. This allows you to use modules stored in remote Git repositories within your configuration.

- **Terraform with AWS (4 questions)**

  - How do you create an EC2 instance with Terraform?

    To create an EC2 instance with Terraform, you define an AWS provider and a resource block for the EC2 instance in your configuration. The configuration specifies the required parameters like the instance type, AMI ID, and other configurations such as security groups and key pairs. After applying the configuration, Terraform will automatically create the EC2 instance for you. by defining the required settings and running terraform apply, Terraform manages the provisioning of the EC2 instance based on the configuration.

  - What are the required fields for defining a VPC in Terraform?

    to define a VPC in Terraform, the required fields typically include the cidr_block, which specifies the IP address range for the VPC, and the enable_dns_support and enable_dns_hostnames options, which determine DNS functionality within the VPC,and while not strictly required, it is common to include other optional parameters tags (for labeling the VPC). These fields help define the basic structure of a VPC in your configuration.

  - Explain how Terraform manages IAM policies in AWS.

    Terraform manages IAM policies in AWS by allowing you to define them as part of your infrastructure configuration. you can either directly write the policy details within the Terraform file or link to an external JSON file. When you run your Terraform setup, it will automatically create or update the IAM policies in AWS and apply them to the right users, roles, or groups. This helps keep everything organized, consistent, and versioned, making it easier to manage access controls across your environment.

- How do you use Terraform to provision and attach an Elastic Load Balancer?

  define the necessary resources in your configuration. This typically includes the aws_lb resource for creating the load balancer, along with aws_lb_target_group for the target group, and aws_lb_listener for setting up listeners that route traffic. You then attach the target group to your instances or services. once configured, running terraform apply will provision the ELB and set up the necessary attachments automatically.

- **Debugging & Error Handling (4 questions)**

  - What does the `terraform validate` command do?

    The terraform validate command checks your Terraform configuration files for errors. It makes sure your code is structured correctly and follows Terraform's rules, but it doesn't touch any resources or make any changes. It's a quick way to ensure that your configuration is error-free before you proceed with planning or applying changes to your infrastructure.

  - How can you debug Terraform errors effectively?

    To debug Terraform errors, enable detailed logging with TF_LOG=DEBUG to get more information. use terraform plan to preview changes and spot issues before applying. Review error messages, check the Terraform documentation, and inspect the state for more insights. Applying your configuration in smaller parts can also help identify problems.

  - What is Terraform's `ignore_changes` lifecycle policy used for?

    The ignore_changes lifecycle policy in terraform is used to prevent certain resource attributes from being modified or updated by Terraform. When applied, it tells terraform to ignore changes to specified attributes, even if they are different from the configuration. this is useful when you want to allow manual changes or external updates to a resource without terraform attempting to overwrite them during the next apply.

  - How do you import existing AWS infrastructure into Terraform?

    To import existing aws infrastructure into terraform, you first define the resource in your terraform configuration without applying it. Then, use the terraform import command along with the resource type and aws identifier to link the existing resource to the configuration. After importing, run terraform plan to review any differences and update the configuration as needed to match the actual state of the resource.